

DSLS PROGRAMMING 5: LECTURE 1

MARTIJN HERBER

Created: 2025-09-11 do 11:17

1. PROGRAMMING 5: DISTRIBUTED COMPUTING

1.1. MARTIJN HERBER / MASTER DATA SCIENCE FOR THE LIFE SCIENCES

1.1.1. (C) 2025 STICHTING HANZEHOGESCHOOL GRONINGEN

1.1.2. LICENSE: CC-BY-NC-SA 4.0 INTERNATIONAL

2. COURSE OBJECTIVES

1. Understand the principles of parallel programming on large clusters (SLURM)
2. Use the Message Passing Interface (MPI) protocol to communicate over a network
3. Be able to use the Apache Spark framework to process large datasets, larger than fit on one computer.
4. Understand the performance of your program, and structure data and operations in such a way that you maximize throughput.

3. SECONDARY OBJECTIVES

1. Choose a proper datastore for Big Data (SQL, Cassandra)
2. Use the SLURM scientific workload manager
3. Learn enough Bash Programming to use clusters
4. Understand big-data algorithms using big-O formalism

4. COMPUTER CLUSTERS

- To carry out "big" computation (long time, lots of memory) you can:
 1. Build a "big" supercomputer (lots of memory, fast processors)
 2. Connect lots of "small" computers together
- We are going to focus on the second option!

5. SLURM: WHAT DOES IT DO?

- Runs programs on one or more multiple computers at once
- "Disconnects" your program from a terminal (manages I/O)
- Distributes work evenly and fairly accross a cluster for many users

6. WHAT DOES SLURM NOT DO?

- Handle communication between parts of your program (processes)
- Restart parts of your program that crash
- Give access to storage by default

7. HOW TO USE SLURM

- Use a command-line program called `srun` to define and submit a job on the command line
- Write a "shell script" to define your job and submit it using `sbatch`
- The latter is preferable for reasons of reproducibility!

8. SHELL SCRIPTS

- The shell is scriptable; ie. you can put a bunch of commands together in a file and execute them in sequence
- This is exactly like typing them on the command line!

9. SHELL PROGRAMMING

- The shell also offers some facilities for "real" programming; variables, loops etc.
- The first such facility you need is... Comments!
- You instruct the SLURM scheduler using comments in your shell script

10. EXAMPLE

```
#!/bin/bash
#SBATCH --job-name=martijn_mapping_brazil
#SBATCH --output=martijn_mapping.out
#SBATCH --nodes=1
#SBATCH --ntasks=16
bowtie2 -t 16 -x /data/datasets/BOWTIE_INDEX/GRCh38_noalt_decoy_as -1 /data/datasets
```

11. LIMITS ON RESOURCES

- SLURM is a scheduler; it manages resources fairly.
- To do so, you help by requesting resources accurately.
- This helps you; your jobs start quicker
- It's also a hard boundary; your job will get killed if you exceed it!

12. SPECIFYING LIMITS

- You specify the limits using the following directives
- `nodes` : how many physical computers you need
- `ntasks-per-node` : how many processes per node you need
- `mem` : how much memory your processes use per process
- `time` : how much time your job gets to run*

13. FINDING OUT ABOUT THE SLURM CLUSTER

- The commands `sinfo` and `sacct` tell you about the status of the system:

```
sinfo
```

PARTITION	AVAIL	TIMELIMIT	NODES	STATE	NODELIST
workstations	up	1-00:00:00	49	down*	nuc002.bin.bioinf.nl,nuc005.bin.bioinf.n
workstations	up	1-00:00:00	37	idle	nuc100.bin.bioinf.nl,nuc101.bin.bioinf.n
assemblix*	up	3-00:00:00	2	idle	assemblix[2019,2025]

14. FINDING OUT ABOUT YOUR ACCOUNT

```
sacct -a
```

JobID	JobName	Partition	Account	AllocCPUS	State	ExitCode
1587	scbert_fi+	asemblix	kcbbe	16	PENDING	0:0
1588	scbert_fi+	asemblix	kcbbe	16	PENDING	0:0
1589	scbert_fi+	asemblix	kcbbe	16	PENDING	0:0
1585	scbert_fi+	asemblix	kcbbe	16	PENDING	0:0

15. FINDING OUT ABOUT YOUR JOBS

- The `squeue` command will tell you what your jobs are doing
- If they don't start in reasonable time, contact Peter or Marcel; something is wrong with the system
- If they immediately return on running, you've made an error; check out the error log SLURM spits out for you!
- NB the `scancel` command will stop one of your running jobs.

16. PARTITIONS

- SLURM can manage different kinds of nodes
- e.g. on the BIN network, there are "workstations" and "asemblix" (big servers)
- The `scontrol show partitions` tells you everything you need to know about them! (Try it!)

17. ESTIMATING RESOURCES

- **Always** provide a limit on the time; base it on a small sample run (or previous runs if available)
- **Hard limits** are (currently) 3 days for assemblix* and 1 day for workstations
- For memory, remember it is the maximum memory
- A good "guesstimate" for many algorithms is 3-5 times your input data, or an index
- Make sure to always ask for $n+1$ number of CPU's (ntasks) to take scripts into account

18. SHELL SCRIPTING FOR SLURM

- Shell "batch" scripts give you a single place to organize a parallel run
- The commands in it are run one by one on the cluster, within the limits
- You can run multiple "jobs" using `srun` **within** your batch file
- You submit the script using `sbatch`

19. SHELL SCRIPTING FOR SLURM: PROGRAMMING

- You can program the bash shell, and thus also batch jobs for SLURM
- This makes it easier to generalize scripts, and for reproducibility
- Finally, it is easier to express repetitive steps
- Rule of thumb; if it takes longer than 10 lines, use python!
- I'm going to give you the bare essentials to get started bash programming, for SLURM sbatch

20. SHELL SCRIPTING: VARIABLES

- Setting variables in bash is dead simple; you just use it!
- Using a variable requires you to prepend "\$"
- NB: OK, one nitpick: don't use spaces around the "="

```
name="martijn"  
echo Hello $name
```

```
Hello martijn
```


21. SHELL SCRIPTING: VARIABLE TYPES

- In principle they are all text
- Alle command line programs expect text
- Sometimes you want to program the shell using other types
- You can make them into integers, or lists (or dictionaries) using `declare`

22. DECLARE SYNTAX: INTEGERS

```
declare -i x  
declare -i y  
declare -i z  
x=1  
y=2  
z=$((x+y))  
echo $z
```

```
3
```

23. DECLARE SYNTAX: LISTS

```
array=(a b c)  
echo Third letter of the alphabet: ${array[2]}
```

```
Third letter of the alphabet: c
```

24. HOW TO USE LISTS: ITERATION!

- You can easily iterate in bash scripts with the usual "while" and "for" constructs
- The syntax is very picky, about spaces
- You can iterate over pre-made lists, or output from commands

```
letters=(D S L S)
for letter in ${letters[@]} ; do echo $letter ! ; done
for f in /commons/Themas/Thema12/HPC/*.fastq ; do ls -hl $f ; done
```

```
D !
S !
L !
S !
-rw-r--r-- 1 martijn staff 26K mei 24 2022 /commons/Themas/Thema12/HPC/part.fastq
-r--r--r-- 1 martijn staff 3.4G mei 1 2017 /commons/Themas/Thema12/HPC/rnaseq.fast
-rw-r--r-- 1 martijn staff 63M mei 21 2022 /commons/Themas/Thema12/HPC/rnaseq_selec
```

25. SYNTAX FOR MAKING RANGES

- bash also offers shortcuts for making ranges

```
for i in {a..f}; do echo file_${i} ; done  
for i in {1..10..2}; do echo fastq_multiplexed_R${i}.fastq ; done
```

```
file_a  
file_b  
file_c  
file_d  
file_e  
file_f  
fastq_multiplexed_R1.fastq  
fastq_multiplexed_R3.fastq  
fastq_multiplexed_R5.fastq  
fastq_multiplexed_R7.fastq  
fastq_multiplexed_R9.fastq
```

26. REDIRECTION

- SLURM by default captures everything your script's contents produce to `stdin` and `stdout`
- You can change this setting using the SLURM options `#SBATCH --output` and `#SBATCH --error`
- -Or- you can redirect using the standard bash redirection operators (`>`, `2>` etc.)

27. EXTENDED EXAMPLE

```
#!/bin/bash -l

#SBATCH --job-name=desiree_kraken2          # Name for your job
#SBATCH --comment="Running Kraken2 against PlusPFP database"

#SBATCH --account=martijn                   # Project account to run your job under
#SBATCH --partition=assemblix               # Partition to run your job on

#SBATCH --output=%x_%j.out                  # Output file
#SBATCH --error=%x_%j.err                   # Error file

#SBATCH --mail-user=m.herber@pl.hanze.nl    # Email username to notify
#SBATCH --mail-type=ALL                     # Type of email notifications to send

#SBATCH --time=0-05:05:00                   # Time limit
#SBATCH --nodes=1                           # How many nodes to run on
#SBATCH --ntasks=2                           # How many tasks per node
#SBATCH --cpus-per-task=65                   # Number of CPUs per task
#SBATCH --mem-per-cpu=200g                   # Memory per CPU
KRAKEN_CONDA="/homes/martijn/org/onderwijs/BIN/Metagenomics/soil_metagenomics/analys
KRAKEN_DB="/commons/Themas/Thema07/metagenomics/INDEXES/RDP11.5/16S_RDP_k2db/"
DATA_DIR=/homes/martijn/org/onderwijs/BIN/Metagenomics/soil_metagenomics_16S/ariana_
BASE_WORKING_DIR=/homes/martijn/org/onderwijs/BIN/Metagenomics/soil_metagenomics/ana
mkdir -p $BASE_WORKING_DIR

source /commons/conda/conda_load.sh
conda activate $KRAKEN_CONDA

barcodes=(barcode01 barcode02 barcode03 barcode04 barcode05 barcode06 barcode07 barc
for bc in "${barcodes[@]}" ;
do
BARCODE_DIR=${BASE_WORKING_DIR}/${bc}
mkdir -p $BARCODE_DIR
kraken2 --db $KRAKEN_DB --threads 64 --report ${BARCODE_DIR}/${bc}_kraken2.report --
bracken -d $KRAKEN_DB -i ${BARCODE_DIR}/${bc}_kraken2.report -r 250 -l S -t 10 -o ${
bracken -d $KRAKEN_DB -i ${BARCODE_DIR}/${bc}_kraken2.report -r 250 -l G -t 10 -o ${
alpha_diversity.py -f ${BARCODE_DIR}/${bc}_kraken2.species.bracken -a Sh > ${BARCODE
done
beta_diversity.py -i ${BASE_WORKING_DIR}/*/*.species.bracken --type bracken --level
beta_diversity.py -i ${BASE_WORKING_DIR}/*/*.genus.bracken --type bracken --level G
```

28. EXERCISE #1

- Your turn!
- Write a SLURM batch script that is run on the "workstations" partition.
- Give an estimated running time of 5 minutes.
- Use the `hostname` shell command to capture the actual host the program is run on.
- Modify the program to run a Python program which sleeps a random amount of time
- (Via a command line argument)

29. SOLUTION #1

- The shell script

```
#!/bin/bash
#SBATCH --partition=workstations
#SBATCH --time=00:05:00
hostname
```

30. SOLUTION #1

```
import sys
from random import randint
from time import sleep
if len(sys.argv) == 2:
    sleep(sys.argv[1])
else:
    sleep(randint(10))
```

31. BASH... A REAL PROGRAMMING LANGUAGE

- Bash is an incredibly large programming language
- Some options include:
 1. String editing
 2. Associative arrays
 3. job control
- However, remember the rule of thumb: "If it's more than 10 lines... use Python!"

Speaker notes